

1. Clase DriverManager y Connection

Para trabajar con bases de datos en Java, necesitamos establecer una conexión. Esto se hace usando dos elementos principales:

- `DriverManager`
- `Connection`

DriverManager

Se encarga de gestionar los drivers y establecer conexiones.

Connection

Representa la conexión activa con la base de datos.

2. Establecer una conexión

Ejemplo básico

```
import java.sql.Connection;
import java.sql.DriverManager;

public class Main {
    public static void main(String[] args) {

        try {
            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/prueba",
                "root",
                "1234"
            );

            System.out.println("Conexión correcta");

            conn.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
}  
}
```

Puntos importantes

- La URL debe ser correcta
 - El usuario y contraseña deben existir
 - Siempre cerrar la conexión
-

3. Manejo de excepciones

Al trabajar con bases de datos, es obligatorio manejar errores.

SQLException

Es la excepción principal en JDBC.

```
catch (SQLException e) {  
    System.out.println(e.getMessage());  
}
```

Métodos útiles

- `getMessage()` → mensaje de error
 - `getErrorCode()` → código del error
 - `getSQLState()` → estado SQL
-

4. Crear un Statement

Antes de ejecutar SQL, necesitamos un objeto `Statement`.

```
Statement stmt = conn.createStatement();
```

Este objeto permite ejecutar instrucciones SQL.

5. Ejecutar consultas (SELECT)

Para consultas que devuelven datos se usa:

```
ResultSet rs = stmt.executeQuery("SELECT * FROM usuarios");
```

- Devuelve un `ResultSet`
 - Contiene los resultados de la consulta
-

6. Ejecutar modificaciones (INSERT, UPDATE, DELETE)

Para operaciones que modifican datos:

```
int filas = stmt.executeUpdate("INSERT INTO usuarios VALUES (3, 'Luis')");
```

Qué devuelve

- Número de filas afectadas
-

7. Ejemplo completo

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class Main {
    public static void main(String[] args) {

        try {
            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/prueba",
                "root",
                "1234"
            );

            Statement stmt = conn.createStatement();

            // INSERT
            stmt.executeUpdate("INSERT INTO usuarios VALUES (4, 'Laura')");

            // UPDATE
```

```
stmt.executeUpdate("UPDATE usuarios SET nombre='Carlos' WHERE id=1");

// DELETE
stmt.executeUpdate("DELETE FROM usuarios WHERE id=2");

conn.close();

} catch (Exception e) {
    e.printStackTrace();
}
}
```

8. Buenas prácticas básicas

- Cerrar siempre la conexión (`close()`)
 - Evitar dejar conexiones abiertas
 - Manejar excepciones correctamente
 - No usar datos hardcodeados en proyectos reales (usuario, contraseña)
-

9. Errores comunes de alumnos

- No cerrar la conexión
- Escribir mal la URL
- No crear la base de datos previamente
- Ejecutar un `SELECT` con `executeUpdate()`
- Olvidar el `WHERE` en un `DELETE` o `UPDATE`

EJERCICIOS

1. Crear un programa Java que se conecte a la base de datos mostrando un mensaje si la conexión es correcta y cerrar correctamente la conexión.
2. Insertar un registro en la tabla `usuarios` usando `Statement`.
3. Actualizar un registro con `Statement`.
4. Eliminar un registro con `Statement`.
5. Ejecutar varios `INSERT` seguidos en el mismo programa.
6. Mostrar el número de filas afectadas por un `UPDATE`.
7. Crear un programa que:
 - a. Cree una tabla desde Java
 - b. Inserte datos
 - c. La elimine al final
8. Crear un menú en consola:
 - i. Insertar
 - ii. Mostrar
 - iii. Salir(usando `Statement`)
9. Crear un programa que detecte si una tabla existe y, si no, la cree.
10. Ejecutar múltiples operaciones y mostrar por pantalla:
 - a. Qué operación se ejecuta
 - b. Cuántas filas afecta